

Working with Dates

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve price. Runs only if the class finishes 01-30. Note that this is the only coverage of outline IV.F, JavaScript dates.

A Date Is One Number

- Under the hood: **milliseconds since Jan 1, 1970 UTC** (the Unix epoch)

```
let now = new Date();           // current date and time
let epoch = new Date(0);        // Jan 1, 1970 UTC

let birthday = new Date(2026, 6, 7); // July 7 - month 6 = July!
let fromString = new Date("2026-07-07"); // ISO, unambiguous
let withTime = new Date("2026-07-07T14:30:00");
```

- **Months are zero-based**, the most common `Date` mistake

Timestamps

```
Date.now();           // 1783382400000 - no object needed  
new Date().getTime(); // same value from an instance
```

- Ideal for measuring elapsed time

```
let start = Date.now();  
// ...do work...  
let elapsedMs = Date.now() - start;
```

Reading the Parts

- `get*` methods read components in **local time**

```
let d = new Date("2026-07-07T14:30:15");  
d.getFullYear(); // 2026  
d.getMonth();    // 6   (July - zero-based!)  
d.getDate();     // 7   (day of MONTH, 1-based)  
d.getDay();      // 2   (day of WEEK, 0 = Sunday)  
d.getHours();    // 14
```

- Watch the mix-up: `getDate()` = day of month, `getDay()` = day of week

Changing Parts (Dates Mutate)

```
let d = new Date("2026-07-07");  
d.setFullYear(2027);  
d.setMonth(11); // December  
d.setDate(25);
```

- `set*` mutates in place; clone first to keep the original

```
let original = new Date("2026-07-07");  
let copy = new Date(original); // independent clone  
copy.setDate(14); // original untouched
```

Date Arithmetic

- `setDate` handles overflow for you; no month-length math

```
function addDays(date, n) {  
  const copy = new Date(date); // clone, don't mutate the input  
  copy.setDate(copy.getDate() + n);  
  return copy;  
}  
  
const jan30 = new Date(2026, 0, 30);  
addDays(jan30, 5).toDateStr(); // "Wed Feb 04 2026"
```

- Rolling past the end of a month just works

Formatting for People

```
let d = new Date("2026-07-07T14:30:00");
d.toLocaleDateString(); // "7/7/2026"
d.toLocaleTimeString(); // "2:30:00 PM"

d.toLocaleDateString("en-US", {
  weekday: "long", year: "numeric",
  month: "long", day: "numeric",
}); // "Tuesday, July 7, 2026"
```

- Build a reusable `Intl.DateTimeFormat` when formatting many dates

Date Math and ISO Strings

- Subtract dates for a gap in milliseconds, then scale

```
let start = new Date("2026-07-01");  
let end = new Date("2026-07-07");  
let diffDays = (end - start) / (1000 * 60 * 60 * 24); // 6
```

- Store/transmit as **ISO 8601**; format for display only at the end

```
new Date("2026-07-07T14:30:00Z").toISOString();  
// "2026-07-07T14:30:00.000Z" (Z = UTC)
```

- Date-only string parses as **UTC**; date-time without **Z** parses as **local**

Looking Ahead: **Temporal**

- **Date** mutates, has zero-based months, and no time-zone support
- **Temporal** fixes all three, finished at TC39, expected in **ES2027**

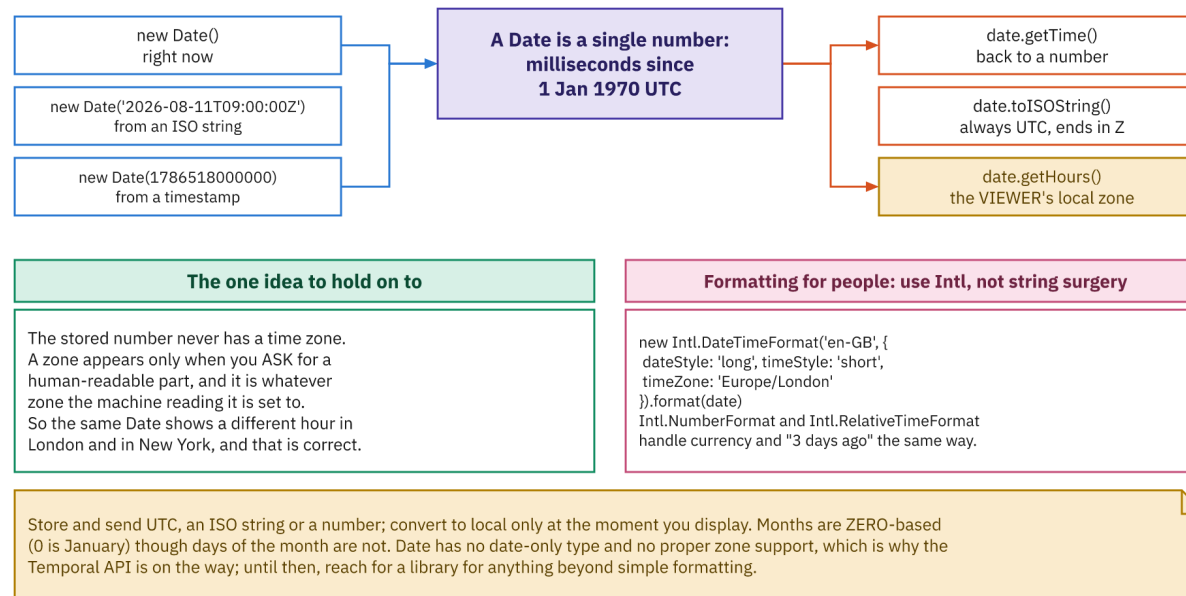
```
// Immutable, 1-based months, explicit time zones
Temporal.PlainDate.from("2026-07-07").add({ days: 5 });
```

- **Not in Node yet**, and not in every browser, a polyfill is required today
- Know it exists and what it fixes; keep using **Date** for now

One Number, Many Faces

The Date and Time Model

A Date is one number. Everything else (the year, the string, the time zone) is presentation.



- A `Date` is milliseconds since 1970; the rest is presentation
- Store UTC, convert to local only when you display